



## Spring Boot-Hibernate Example

### demo1-boot-hibernate: Files required

|                           |                           |
|---------------------------|---------------------------|
| 1. MyBootApplication.java | 2. JLCAppConfig.java      |
| 3. Customer.java          | 4. CustomerDAO.java       |
| 5. CustomerDAOImpl.java   | 6. application.properties |
| 7. pom.xml                |                           |

#### 1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    public static void main(String[] args) {
        SpringApplication.run(MyBootApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust = new Customer("sri", "sri@jlc", 55555, "Blore");
        custDAO.addCustomer(cust);

        // 2.get All Customers
        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");
    }
}
```



### // 3.Get Customers By City

```
List<Customer> mylist2 = custDAO.getCustomersByCity("Blore");
mylist2.forEach(mycust -> System.out.println(mycust));

System.out.println("Done!!!");
}
}
```

## 2)JLAppConfig.java

```
package com.jlcindia.springboot;

import java.util.Properties;
import javax.sql.DataSource;
import org.hibernate.SessionFactory;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class JLAppConfig {

    @Bean
    public LocalSessionFactoryBean getSessionFactory(DataSource mydataSource) {
        Properties myprops = new Properties();
        myprops.put("hibernate.show_sql","true");
        myprops.put("hibernate.dialect","org.hibernate.dialect.MySQL5Dialect");
        //myprops.put("hibernate.hbm2ddl.auto","update");

        LocalSessionFactoryBean mysessionFactory = new LocalSessionFactoryBean();

        mysessionFactory.setDataSource(mydataSource); //1
        mysessionFactory.setHibernateProperties(myprops); //2
        mysessionFactory.setPackagesToScan("com.jlcindia.springboot"); //3
        return mysessionFactory;
    }

    @Bean
    public HibernateTemplate getHT(SessionFactory mysessionFactory) {
        return new HibernateTemplate(mysessionFactory);
    }
}
```



@Bean

```
public HibernateTransactionManager getTxManager(SessionFactory sessionFactory) {  
    return new HibernateTransactionManager(sessionFactory);  
}  
}
```

### 3) Customer.java

```
package com.jlcindia.springboot;  
  
import javax.persistence.*;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
@Entity  
@Table(name="mycustomers")  
public class Customer {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name="cid")  
    private int cid;  
  
    @Column(name="cname")  
    private String cname;  
  
    @Column(name="email")  
    private String email;  
  
    @Column(name="phone")  
    private int phone;  
  
    @Column(name="city")  
    private String city;  
  
    public Customer() {}  
    public Customer(String cname, String email, int phone, String city) {  
        this.cname = cname;  
        this.email = email;  
        this.phone = phone;  
        this.city = city;  
    }  
  
    public Customer(int cid, String cname, String email, int phone, String city) {  
        super();  
        this.cid = cid;  
        this.cname = cname;
```



```
        this.email = email;
        this.phone = phone;
        this.city = city;
    }

    //Setters and Getters
    //toString()
}
```

#### **4) CustomerDAO.java**

```
package com.jlcindia.springboot;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {
    public void addCustomer(Customer cust);
    public List<Customer> getAllCustomers();
    public List<Customer> getCustomersByCity(String city);
}
```

#### **5) CustomerDAOImpl.java**

```
package com.jlcindia.springboot;

import java.util.List;

import org.hibernate.criterion.DetachedCriteria;
import org.hibernate.criterion.Restrictions;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @Autowired
    HibernateTemplate htemp;
```



```
@Override
public void addCustomer(Customer cust) {
    htemp.save(cust);
}

@Override
public List<Customer> getAllCustomers() {

    List<Customer> mylist = htemp.loadAll(Customer.class);
    return mylist;

}

@Override
public List<Customer> getCustomersByCity(String city) {
    /*
    String HQL = "from Customer cust where cust.city=?";
    List<Customer> mylist =(List<Customer>) htemp.find(HQL, city);
    */

    DetachedCriteria dc=DetachedCriteria.forClass(Customer.class);
    dc=dc.add(Restrictions.eq("city",city));
    List<Customer> mylist = (List<Customer>) htemp.findByCriteria(dc);
    return mylist;
}

}
```

## **6)application.properties**

### **#Database Properties**

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

### **#Log Properties**

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```



### 7)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo1-boot-hibernate</artifactId>
<version>1.0</version>
<name>demo1-boot-hibernate</name>
<description>This is Spring Boot and Hibernate Application</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



## Spring Boot-JPA Example

### demo2-boot-jpa: Files required

|                           |                         |
|---------------------------|-------------------------|
| 1. MyBootApplication.java | Same as Demo1           |
| 2. Customer.java          | Same as Demo1           |
| 3. CustomerDAO.java       | Same as Demo1           |
| 4. CustomerDAOImpl.java   | <b>Updated in Demo1</b> |
| 5. JLCAppConfig.java      | <b>Updated in Demo1</b> |
| 6. application.properties | Same as Demo1           |
| 7. pom.xml                | Same as Demo1           |

#### 4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;
```

```
import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```

```
@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {
```

```
@PersistenceContext
EntityManager entityManager;
```

```
@Override
public void addCustomer(Customer cust) {
    entityManager.persist(cust);
}
```

```
@Override
public List<Customer> getAllCustomers() {
    String jpaQL= "from Customer cust";
    List<Customer> mylist = entityManager.createQuery(jpaQL, Customer.class).getResultList();
    return mylist;
}
```



```
@Override
public List<Customer> getCustomersByCity(String city) {
String jpQL= "from Customer cust where cust.city=?1";
List<Customer> mylist =
entityManager.createQuery(jpQL, Customer.class)
.setParameter(1, city)
.getResultList();

return mylist;
}
}
```

#### **6)JLAppConfig.java**

```
package com.jlcindia.springboot;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JLAppConfig {

}
```



## Spring JPA

### EntityManager IMP Methods

|     |                                                                                                  |
|-----|--------------------------------------------------------------------------------------------------|
| 1.  | <code>void persist(Object entity)</code>                                                         |
| 2.  | <code>&lt;T&gt; T merge(T entity)</code>                                                         |
| 3.  | <code>void remove(Object entity)</code>                                                          |
| 4.  | <code>&lt;T&gt; T getReference(Class&lt;T&gt; entityClass, Object primaryKey)</code>             |
| 5.  | <code>&lt;T&gt; T find(Class&lt;T&gt; entityClass, Object PK)</code>                             |
| 6.  | <code>&lt;T&gt; T find(Class&lt;T&gt; entityClass, Object PK, LockModeType lockMode)</code>      |
| 7.  | <code>Query createQuery(String qlStr)</code>                                                     |
| 8.  | <code>&lt;T&gt; TypedQuery&lt;T&gt; createQuery(String qlStr, Class&lt;T&gt; resultClass)</code> |
| 9.  | <code>Query createNamedQuery(String name)</code>                                                 |
| 10. | <code>&lt;T&gt; TypedQuery&lt;T&gt; createNamedQuery(String nm, Class&lt;T&gt; resCls)</code>    |
| 11. | <code>Query createNativeQuery(String qlStr)</code>                                               |
| 12. | <code>Query createNativeQuery(String qlStr, Class resultClass)</code>                            |
| 13. | <code>&lt;T&gt; TypedQuery&lt;T&gt; createQuery(CriteriaQuery&lt;T&gt; criteriaQuery)</code>     |
| 14. | <code>void refresh(Object entity)</code>                                                         |
| 15. | <code>boolean contains(Object entity)</code>                                                     |
| 16. | <code>EntityTransaction getTransaction()</code>                                                  |
| 17. | <code>void detach(Object entity)</code>                                                          |
| 18. | <code>void clear()</code>                                                                        |
| 19. | <code>void flush()</code>                                                                        |
| 20. | <code>void close()</code>                                                                        |



## Spring Boot-JPA Complete Example

### demo3-boot-jpa: Files required

|                           |                         |
|---------------------------|-------------------------|
| 1. MyBootApplication.java | <b>Updated in demo3</b> |
| 2. Customer.java          | Same as demo2           |
| 3. CustomerDAO.java       | <b>Updated in demo3</b> |
| 4. CustomerDAOImpl.java   | <b>Updated in demo3</b> |
| 5. application.properties | <b>Updated in demo3</b> |
| 6. pom.xml                | Same as demo2           |

#### 1)MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust1 = new Customer("vas", "vas@jlc", 12345, "Blore");
        custDAO.addCustomer(cust1);

        // 2.Update Customer
        Customer cust2 = new Customer(8,"SD", "SD@jlc", 999, "Delhi");
        custDAO.updateCustomer(cust2);

        //3. Get Customer By Cid
        Customer cust3 =custDAO.getCustomerByCid(8);
        System.out.println("custt 3: "+cust3);
    }
}
```



```
Customer cust4 = custDAO.getCustomerByCid(88);  
System.out.println("custt 4: "+cust4);
```

#### //4. Delete Customer By Cid

```
custDAO.deleteCustomer(1);  
custDAO.deleteCustomer(101);
```

#### //5.All Customers - JPAQL

```
List<Customer> mylist1 = custDAO.getAllCustomers();  
for(Customer mycust: mylist1) {  
    System.out.println(mycust);  
}
```

#### //6.All Customers – Native Query

```
List<Customer> mylist2 = custDAO.getAllCustomers1();  
for(Customer mycust: mylist2) {  
    System.out.println(mycust);  
}
```

#### //7. Customers By City

```
List<Customer> mylist3 = custDAO.getCustomersByCity("Hyd");  
for(Customer mycust: mylist3) {  
    System.out.println(mycust);  
}
```

#### //8. Customers By Phone

```
List<Customer> mylist4 = custDAO.getCustomersByPhone(12345);  
for(Customer mycust: mylist4) {  
    System.out.println(mycust);  
}
```

#### //9. Customers By Email

```
Customer cust5 = custDAO.getCustomerByEmail("hello@jlc");  
System.out.println(cust5);
```

#### //10. Customers By Email and Phone

```
List<Customer> mylist5= custDAO.getCustomersByEmailAndPhone("hello@jlc", 12345);  
for(Customer mycust: mylist5) {  
    System.out.println(mycust);  
}
```



**//10. Customers By Email and Phone**

```
List<Customer> mylist6 = custDAO.getCustomersByEmailAndPhone("hai@jlc", 12345);  
for(Customer mycust: mylist6) {  
    System.out.println(mycust);  
}
```

**//11. Customers By Email Or Phone**

```
List<Customer>mylist7 = custDAO.getCustomersByEmailOrPhone("hai@jlc", 12345);  
for(Customer mycust: mylist7) {  
    System.out.println(mycust);  
}
```

**//12. Customers By Email Or Phone**

```
List<Customer> mylist8 = custDAO.getCustomers("Blore", "hello@jlc", 12345);  
for(Customer mycust: mylist8 ) {  
    System.out.println(mycust);  
}
```

**//13. Customers By NameContains**

```
List<Customer> mylist9= custDAO.getCustomersByNameContains("sri");  
for(Customer mycust: mylist9) {  
    System.out.println(mycust);  
}
```

**//13. Customers By NameContains**

```
List<Customer> mylist10 = custDAO.getCustomersByNameContains("vas");  
for(Customer mycust: mylist10 ) {  
    System.out.println(mycust);  
}
```

**//14. Customers Count**

```
System.out.println(custDAO.getCustomerCount());
```

**//15. Customers Count By City**

```
System.out.println(custDAO.getCustomerCountByCity("Blore"));  
System.out.println(custDAO.getCustomerCountByCity("Hyd"));
```

**//16. Phone By Email**

```
System.out.println(custDAO.getCustomerPhoneByEmail("hello@jlc"));
```

**//17. City By Email**

```
System.out.println(custDAO.getCustomerCityByEmail("hello@jlc"));
```



#### //18. Email and Phones of Customers By city

```
Map<String,Integer> mymap= custDAO.getCustomersInfo("Blore");
System.out.println(mymap);
System.out.println("Done!!!");
}

public static void main(String[] args) {
SpringApplication.run(MyBootApplication.class, args);
}
}
```

### 3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.math.BigInteger;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {

    public void addCustomer(Customer cust);
    public void updateCustomer(Customer cust);
    public void deleteCustomer(int cid);
    public Customer getCustomerByCid(int cid);

    public List<Customer> getAllCustomers();
    public List<Customer> getAllCustomers1();
    public List<Customer> getCustomersByCity(String city);
    public List<Customer> getCustomersByPhone(int phone);

    public Customer getCustomerByEmail(String email);

    public List<Customer> getCustomersByEmailAndPhone(String email,int phone);
    public List<Customer> getCustomersByEmailOrPhone(String email,int phone);
    public List<Customer> getCustomers(String city,String email,int phone);
    public List<Customer> getCustomersByNameContains(String name);

    public BigInteger getCustomerCount();
    public BigInteger getCustomerCountByCity(String city);
    public int getCustomerPhoneByEmail(String email);
    public String getCustomerCityByEmail(String email);
    public Map<String,Integer> getCustomersInfo(String city);
}
```



#### 4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.math.BigInteger;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.Query;

import org.hibernate.dialect.Database;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @PersistenceContext
    EntityManager entityManager;

    @Override
    public void addCustomer(Customer cust) {
        entityManager.persist(cust);
    }

    @Override
    public void updateCustomer(Customer cust) {
        entityManager.merge(cust);
    }

    @Override
    public void deleteCustomer(int cid) {
        Customer cust =entityManager.find(Customer.class, cid);
        if(cust!=null)
            entityManager.remove(cust);
    }
}
```



```
@Override
public Customer getCustomerByCid(int cid) {
// Customer cust = entityManager.getReference(Customer.class, cid);
// System.out.println("In CustomerDAOImpl : "+cust);
Customer cust = entityManager.find(Customer.class, cid);

return cust;
}

@Override
public List<Customer> getAllCustomers(){
String JPAQL = "from Customer cust";
Query query = entityManager.createQuery(JPAQL, Customer.class);
List<Customer> mylist = query.getResultList();
return mylist;
}

@Override
public List<Customer> getAllCustomers1(){

String SQL = "select * from mycustomers cust";
return entityManager.createNativeQuery(SQL, Customer.class).getResultList();

}

@Override
public List<Customer> getCustomersByCity(String city){

String JPAQL = "from Customer cust where cust.city=?1";
return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, city)
                .getResultList();

}

@Override
public Customer getCustomerByEmail(String email) {

String JPAQL = "from Customer cust where cust.email=?1";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .getSingleResult();

}
```



```
@Override
public List<Customer> getCustomersByPhone(int phone){
String JPAQL = "from Customer cust where cust.phone=?1";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, phone)
                .getResultList();
}

@Override
public List<Customer> getCustomersByEmailAndPhone(String email,int phone){
String JPAQL = "from Customer cust where cust.email=?1 and cust.phone=?2";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .setParameter(2, phone)
                .getResultList();
}

@Override
public List<Customer> getCustomersByEmailOrPhone(String email,int phone){
String JPAQL = "from Customer cust where cust.email=?1 or cust.phone=?2";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .setParameter(2, phone)
                .getResultList();
}

@Override
public BigInteger getCustomerCount() {
String SQL = "select count(*) from mycustomers cust";

return (BigInteger) entityManager.createNativeQuery(SQL) .getSingleResult();
}

@Override
public BigInteger getCustomerCountByCity(String city) {
String SQL = "select count(*) from mycustomers cust where cust.city=?1";

return (BigInteger) entityManager.createNativeQuery(SQL)
                .setParameter(1, city)
                .getSingleResult();
}
```



```
@Override
public int getCustomerPhoneByEmail(String email) {
String SQL = "select phone from mycustomers cust where cust.email=?1";

return (Integer) entityManager.createNativeQuery(SQL)
.setParameter(1, email)
.getSingleResult();
}

@Override
public String getCustomerCityByEmail(String email) {
String SQL = "select city from mycustomers cust where cust.email=?1";

return (String) entityManager.createNativeQuery(SQL)
.setParameter(1, email)
.getSingleResult();
}

@Override
public List<Customer> getCustomers(String city,String email,int phone){
String JPAQL = "from Customer cust where cust.city=?1 and cust.email=?2 and
cust.phone=?3";

return entityManager.createQuery(JPAQL, Customer.class)
.setParameter(1,city)
.setParameter(2, email)
.setParameter(3, phone)
.getResultList();
}

@Override
public Map<String,Integer> getCustomersInfo(String city) {
String SQL = "select email,phone from mycustomers cust where cust.city=?1";

List mylist = entityManager.createNativeQuery(SQL)
.setParameter(1,city)
.getResultList();

Map<String,Integer> mymap= new HashMap<>();
for(Object obj:mylist) {
Object myobj [] = (Object [])obj;
mymap.put((String)myobj[0], (Integer)myobj[1]);
}

return mymap;
}
```



@Override

**public List<Customer> getCustomersByNameContains(String name){**

String JPAQL = "from Customer cust where cust.cname like :pattern";

return entityManager.createQuery(JPAQL, Customer.class)

.setParameter("pattern","%" + name + "%")

.getResultList();

**}**

**}**

## **5)application.properties**

### **#Database Properties**

spring.datasource.driver-class-name=com.mysql.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5

spring.datasource.username=root

spring.datasource.password=srinivas

### **#JPA Properties**

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

### **#Log Properties**

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %m %n



## Questions

**Q36)** What are the Bean Dependencies required for SessionFactory?

**Ans:**

- 1) DataSource
- 2) HibernateProperties
- 3) Mapping Classes

**Q37)** What are the Bean Dependencies required for HibernateTemplate?

**Ans:**

SessionFactory

**Q38)** What are the Bean Dependencies required for HibernateTransactionManager?

**Ans:**

SessionFactory

**Q39)** What are the Bean Dependencies required for EntityManagerFactory?

**Ans:**

- 1) DataSource
- 2) HibernateProperties
- 3) Mapping Classes

**Q40)** What are the Bean Dependencies required for JpaTransactionManager?

**Ans:**

EntityManagerFactory

**Q41)** Is it required to configure EntityManagerFactory and JpaTransactionManager for using JPA with Spring Boot?

**Ans:**

- No Need to Configure EntityManagerFactory and JpaTransactionManager.
- These two will be auto-configured by Spring Boot.

**Q42)** What is the use of JpaVendorAdapter and what are the various JpaVendorAdapters available?

**Ans:**

- JpaVendorAdapter is used to specify the JPA provider
- Interface JpaVendorAdapter.
- SPI interface that allows to plug in vendor-specific behavior into Spring's EntityManagerFactory creators.
- Serves as single configuration point for all vendor-specific properties.
- JpaVendorAdapter implementation classes are
  1. HibernateJpaVendorAdapter
  2. EclipseLinkJpaVendorAdapter

**Q43)** What are the Beans which are AutoConfigured by Spring Boot from the following?

Ans:

You Can Cross check which of the following beans will be AutoConfigured by Spring Boot by Injecting as follows.

|                                                                        |                |
|------------------------------------------------------------------------|----------------|
| @Autowired(required = false)<br>DataSource ds;                         | AutoConfigured |
| @Autowired(required = false)<br>JdbcTemplate jtemp;                    | AutoConfigured |
| @Autowired(required = false)<br>NamedParameterJdbcTemplate njtemp;     | AutoConfigured |
| @Autowired(required = false)<br>EntityManagerFactory emf;              | AutoConfigured |
| @Autowired(required = false)<br>JpaTransactionManager txManager;       | AutoConfigured |
| @Autowired(required = false)<br>SessionFactory sf;                     | No             |
| @Autowired(required = false)<br>HibernateTemplate htemp;               | No             |
| @Autowired(required = false)<br>HibernateTransactionManager txManager; | No             |

**Q44)** How can I do the Persistence Operations with JPA?

Ans:

**EntityManager** can be used to perform Persistence Operations.

**Q45)** How Can I Inject the EntityManager in the required DAO?

Ans:

```
@PersistenceContext
EntityManager entityManager;
```



**Q46) What is the Speciality of @PersistentContext annotation? How it works differently from @Autowired/@Resource?**

**Ans:**

**Q47) What is EntityManager?**

**Ans:**

- Each EntityManager instance is associated with a persistence context.
- Within the persistence context, the entity instances and their lifecycle are managed.
- A persistence context is like a cache which contains a set of persistent entities
- When Transaction is finished, all persistent objects are detached from the EntityManager's persistence context and are no longer managed.

**Q48) Who is the Provider of @PersistentContext ?**

**Ans:** JPA

**Q49) What are the IMP Methods of EntityManager?**

**Ans:** Refer Notes – Page No: 62

**Q50) When Connection Pool will be created in Spring Boot Application?**

**Ans:** At the Application Start-UP

**Q51) When Connection Pool will be created in Spring Application?**

**Ans:** At the Application Start-UP

**Q52) How to Insert the Record in JPA?**

**Ans:**

```
entityManager.persist(cust);
```

**Q53) How to Update the Record in JPA?**

**Ans:**

```
entityManager.merge(cust);
```

**Q54) How to Delete the Record in JPA?**

**Ans:**

```
Customer cust=entityManager.find(Customer.class, cid);  
If(cust!=null)  
    entityManager.remove(cust);
```

**Q55) How to Select the Record by Primary Key in JPA?**

**Ans:**

Two Options :

- 1) Customer cust=entityManager.**getReference**(Customer.class, cid);
- 2) Customer cust=entityManager.**find**(Customer.class, cid);



**Q56) What are the difference between getReference() and find() methods?**

**Ans:**

Refer Video.

**Q57) How to Select the Single Values from Database in JPA?**

**Ans:**

Prefer to Use Native Queries

```
String SQL="select count(*) from mycustomers ";
```

```
BigInteger count= (BigInteger) entityManager.createNativeQuery(SQL).getSingleResult();
```



## Spring Boot – JPA Example using JPA Query Languages

### demo4-boot-jpa: Files required

|                           |                  |
|---------------------------|------------------|
| 1. MyBootApplication.java | Updated in Demo4 |
| 2. Customer.java          | Updated in Demo4 |
| 3. CustomerDAO.java       | Updated in Demo4 |
| 4. CustomerDAOImpl.java   | Updated in Demo4 |
| 5. application.properties | Updated in Demo4 |
| 6. pom.xml                | Same as Demo3    |

#### 1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.slf4j.*;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    static Logger logger = LoggerFactory.getLogger("MyBootApplication");

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        logger.debug("Demo4- run() Starts");
        logger.info("Demo4- run() Starts");
        logger.warn("Demo4- run() Starts");

        // 1. Customers By City1 - JPA-QL
        List<Customer> mylist1 = custDAO.getCustomersByCity1("Blore");
        for (Customer mycust : mylist1) {
            System.out.println(mycust);
        }
    }
}
```



```
logger.info("-----");
// 2. Customers By City2 - Native-QL
List<Customer> mylist2 = custDAO.getCustomersByCity2("Blore");
for (Customer mycust : mylist2) {
    System.out.println(mycust);
}

logger.info("-----");
// 3. Customers By City3 - Named - JPA-QL
List<Customer> mylist3 = custDAO.getCustomersByCity3("Blore");
for (Customer mycust : mylist3) {
    System.out.println(mycust);
}

logger.info("-----");
// 4. Customers By City4 - Named - Native-QL
List<Customer> mylist4 = custDAO.getCustomersByCity4("Blore");
for (Customer mycust : mylist4) {
    System.out.println(mycust);
}

logger.info("-----");
// 5. Customers By City - Criteria Queries
List<Customer> mylist5 = custDAO.getCustomersByCity("Blore");
for (Customer mycust : mylist5) {
    System.out.println(mycust);
}

logger.info("-----");
// 6. Customers By Email - Criteria Queries
List<Customer> mylist6 = custDAO.getCustomersByEmail("sri@jlc");
for (Customer mycust : mylist6) {
    System.out.println(mycust);
}

logger.info("-----");
// 7. Customers By City and Email - Criteria Queries
List<Customer> mylist7 = custDAO.getCustomers("sri@jlc","Blore");
for (Customer mycust : mylist7) {
    System.out.println(mycust);
}
System.out.println("Done!!!");
logger.debug("Demo4- run() ends");
logger.info("Demo4- run() ends");
logger.warn("Demo4- run() ends");
}
```



```
public static void main(String[] args) {

logger.debug("Demo4- main() Starts");
logger.info("Demo4 - main() Starts");
logger.warn("Demo4- main() Starts");

SpringApplication.run(MyBootApplication.class, args);

logger.debug("Demo4- main() Ends");
logger.info("Demo4 - main() Ends");
logger.warn("Demo4- main() Ends");

}
}
```

## 2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Entity
@Table(name="mycustomers")

@NamedQuery(name = "customerByCity-A",
            query = "from Customer cust where cust.city=:mycity")

@NamedQuery(name="customerByCity-B",
            query="select * from mycustomers cust where cust.city=?1",
            resultClass = Customer.class)
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="cid")
    private int cid;

    @Column(name="cname")
    private String cname;

    @Column(name="email")
    private String email;

    @Column(name="phone")
    private int phone;
```



```
@Column(name="city")
private String city;

public Customer() {}
public Customer(String cname, String email, int phone, String city) {
    this.cname = cname;
    this.email = email;
    this.phone = phone;
    this.city = city;
}

public Customer(int cid, String cname, String email, int phone, String city) {
    super();
    this.cid = cid;
    this.cname = cname;
    this.email = email;
    this.phone = phone;
    this.city = city;
}

//Setters and Getters
//toString()
}
```

### 3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {

    public List<Customer> getCustomersByCity1(String city);
    public List<Customer> getCustomersByCity2(String city);
    public List<Customer> getCustomersByCity3(String city);
    public List<Customer> getCustomersByCity4(String city);

    public List<Customer> getCustomersByCity(String city);
    public List<Customer> getCustomersByEmail(String email);
    public List<Customer> getCustomers(String email,String city);
}
```



#### 4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.criteria.CriteriaBuilder;
import javax.persistence.criteria.CriteriaQuery;
import javax.persistence.criteria.Predicate;
import javax.persistence.criteria.Root;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @PersistenceContext
    EntityManager entityManager;

    public List<Customer> getCustomersByCity1(String city){
        String JPAQL = "from Customer cust where cust.city=:mycity";

        List<Customer> mylist = entityManager.createQuery(JPAQL, Customer.class)
        .setParameter("mycity", city)
        .setFirstResult(0)
        .setMaxResults(3)
        .getResultList();

        return mylist;
    }

    public List<Customer> getCustomersByCity2(String city){
        String SQL = "select * from mycustomers cust where cust.city=:mycity";

        List<Customer> mylist = entityManager.createNativeQuery(SQL, Customer.class)
        .setParameter("mycity", city)
        .setFirstResult(0)
        .setMaxResults(3)
        .getResultList();

        return mylist;
    }
}
```



```
public List<Customer> getCustomersByCity3(String city){

return entityManager.createNamedQuery("customerByCity-A")
.setParameter("mycity", city)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();

}

public List<Customer> getCustomersByCity4(String city){

return entityManager.createNamedQuery("customerByCity-B")
.setParameter(1, city)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}

public List<Customer> getCustomersByCity(String mycity){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate cityPredicate = cbuilder.equal(myroot.get("city"), mycity);
mycriteria.where(cityPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();

}
public List<Customer> getCustomersByEmail(String myemail){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), myemail);

mycriteria.where(emailPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}
```



```
public List<Customer> getCustomers(String email,String city){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), email);
Predicate cityPredicate = cbuilder.equal(myroot.get("city"), city);
mycriteria.where(emailPredicate,cityPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}
}
```

### 5)application.properties

#### **#Database Properties**

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#### **#Log Properties**

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n

logging.level.com.zaxxer=DEBUG
logging.level.org.hibernate=INFO
logging.level.org.springframework=INFO
logging.level.com.jlcindia=WARN
```



## Spring Boot - JPA Example using Programatic Transactions

### demo5-boot-jpa: Files required

|                                    |                                       |
|------------------------------------|---------------------------------------|
| 1. MyBootApplication.java          | 2. Account.java                       |
| 3. AccountDAO.java                 | 4. AccountDAOImpl.java                |
| 5. InsufficientFundsException.java | 6. InvalidAccountNumberException.java |
| 7. application.yml                 | 8. pom.xml                            |

#### 1) MyBootApplication.java

```
package com.jlcindia.springboot;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    static Logger logger = LoggerFactory.getLogger("MyBootApplication");

    @Autowired
    AccountDAO accountDAO;

    @Override
    public void run(String... args) throws Exception {

        logger.debug("Demo5- run() Starts");
        logger.info("Demo5- run() Starts");
        logger.warn("Demo5- run() Starts");

        System.out.println(accountDAO.getBalance(101));
        System.out.println(accountDAO.getBalance(102));
        System.out.println(accountDAO.getBalance(103));

        System.out.println("-----");
        System.out.println(accountDAO.getBalance(101));
        accountDAO.deposit(101,5000);
        System.out.println(accountDAO.getBalance(101));
    }
}
```



```
accountDAO.withdraw(101,5000);
System.out.println(accountDAO.getBalance(101));

try {
    System.out.println(accountDAO.getBalance(103));
    System.out.println(accountDAO.getBalance(101));
    accountDAO.fundsTransfer(103,101,5500.0);
    System.out.println(accountDAO.getBalance(103));
    System.out.println(accountDAO.getBalance(101));
} catch (Exception ex) {
    System.out.println(ex);
}

System.out.println("Done!!!");
logger.debug("Demo5- run() ends");
logger.info("Demo5- run() ends");
logger.warn("Demo5- run() ends");

}

public static void main(String[] args) {
    logger.debug("Demo5- main() Starts");
    logger.info("Demo5 - main() Starts");
    logger.warn("Demo5- main() Starts");
    SpringApplication.run(MyBootApplication.class, args);
    logger.debug("Demo5- main() Ends");
    logger.info("Demo5 - main() Ends");
    logger.warn("Demo5- main() Ends");
}

}
```

## 2) Account.java

```
package com.jlcindia.springboot;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
@Entity
@Table(name = "myaccounts")
public class Account {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "accno")
    private int accno;

    @Column(name = "atype")
    private String atype;

    @Column(name = "balance")
    private double balance;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

### 3) AccountDAO.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface AccountDAO {
    public double getBalance(int accno);
    public void deposit(int accno,double amount);
    public void withdraw(int accno,double amount);
    public void fundsTransfer(int saccno,int daccno,double amount);
}
```

### 4) AccountDAOImpl.java

```
package com.jlcindia.springboot;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
*/
@Repository
public class AccountDAOImpl implements AccountDAO {

    @PersistenceContext
    EntityManager entityManager;

    @Autowired
    PlatformTransactionManager txManager;

    @Override
    public double getBalance(int accno) {
        Account account = entityManager.find(Account.class, accno);
        double bal = 0.0;
        if(account!=null)
            bal = account.getBalance();
        else
            throw new InvalidAccountNumberException();

        return bal;
    }

    @Override
    public void deposit(int accno, double amount) {
        TransactionStatus ts=null;
        try {
            TransactionDefinition txDef =
                new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
            ts=txManager.getTransaction(txDef); //Tx Begin

            //Ops Starts
            Account account = entityManager.find(Account.class, accno);
            if(account!=null) {
                account.setBalance(account.getBalance()+amount);
                entityManager.merge(account);
            }else {
                throw new InvalidAccountNumberException();
            }

            txManager.commit(ts); //Tx Ends

        }catch(Exception ex) {
            txManager.rollback(ts); //Tx Ends
        }
    }
}
```



```
@Override
public void withdraw(int accno, double amount) {
    TransactionStatus ts=null;
    try {
        TransactionDefinition txDef =
        new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
ts =txManager.getTransaction(txDef); //Tx Begin

        //Ops Starts
Account account = entityManager.find(Account.class, accno);
        if(account!=null) {
            double currBal = account.getBalance();
            if(currBal>=amount+5000) {
                account.setBalance(currBal-amount);
entityManager.merge(account);
            }else {
                throw new InsufficientFundsException();
            }
        }else {
            throw new InvalidAccountNumberException();
        }
txManager.commit(ts); //Tx Ends

    }catch(Exception ex) {
txManager.rollback(ts); //Tx Ends
    }
}

@Override
public void fundsTransfer(int saccno, int daccno, double amount) {
    TransactionStatus ts=null;
    try {
        TransactionDefinition txDef =
        new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
ts =txManager.getTransaction(txDef); //Tx Begin

        System.out.println(getBalance(daccno));
        deposit(daccno, amount);
        System.out.println(getBalance(daccno));
        withdraw(saccno, amount);

txManager.commit(ts); //Tx Ends
    }catch(Exception ex) {
txManager.rollback(ts); //Tx Ends
    }
}
}
```



#### 5) InvalidAccountNumberException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class InvalidAccountNumberException extends RuntimeException {
}
```

#### 6) InsufficientFundsException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class InsufficientFundsException extends RuntimeException {
}
```

#### 7)application.yml

```
spring:
  datasource:
    driver-class-name: com.mysql.jdbc.Driver
    url: jdbc:mysql://localhost:3306/myjlcdb5
    username: root
    password: srinivas

logging:
  level:
    root: INFO
    com.zaxxer: DEBUG
    org.hibernate: INFO
    org.springframework: INFO
    com.jlcindia: WARN
```

#### 8)pom.xml

```
<project .....>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo5-boot-jpa</artifactId>
```



```
<version>1.0</version>
<name>demo5-boot-jpa</name>
<description>Pragmatic TxS Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



## Spring Boot - JPA Example using Declarative Transactions

### demo6-boot-jpa: Files required

|                                       |                         |
|---------------------------------------|-------------------------|
| 1. MyBootApplication.java             | Same as Demo5           |
| 2. Account.java                       | Same as Demo5           |
| 3. AccountDAO.java                    | Same as Demo5           |
| 4. AccountDAOImpl.java                | <b>Updated in Demo6</b> |
| 5. InsufficientFundsException.java    | Same as Demo5           |
| 6. InvalidAccountNumberException.java | Same as Demo5           |
| 7. application.yml                    | Same as Demo5           |
| 8. pom.xml                            | Same as Demo5           |

#### 4) AccountDAOImpl.java

```
package com.jlcindia.springboot;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class AccountDAOImpl implements AccountDAO {

    @PersistenceContext
    EntityManager entityManager;

    public double getBalance(int accno) {
        Account account = entityManager.find(Account.class, accno);
        double bal = 0.0;
        if(account!=null)
            bal = account.getBalance();
        else
            throw new InvalidAccountNumberException();
        return bal;
    }
}
```



```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void deposit(int accno, double amount) {
```

```
//Ops Starts
Account account = entityManager.find(Account.class, accno);
if(account!=null) {
    account.setBalance(account.getBalance()+amount);
    entityManager.merge(account);
}else {
    throw new InvalidAccountNumberException();
}

}
```

```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void withdraw(int accno, double amount) {
```

```
//Ops Starts
Account account = entityManager.find(Account.class, accno);
if(account!=null) {
    double currBal = account.getBalance();
    if(currBal>=amount+5000) {
        account.setBalance(currBal-amount);
        entityManager.merge(account);
    }else {
        throw new InsufficientFundsException();
    }
}else {
    throw new InvalidAccountNumberException();
}

}
```

```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void fundsTransfer(int saccno, int daccno, double amount) {
```

```
System.out.println(getBalance(daccno));
deposit(daccno, amount);
System.out.println(getBalance(daccno));
withdraw(saccno, amount);

}
}
```



**Q58) How many types of Query Languages supported by JPA?**

**Ans:**

JPA supports 4 types of Query Languages like Hibernate

- 1) JPA QL:
- 2) Native Queries:
- 3) Named Queries:
- 4) Criteria Queries:

**Q59) How to use JPA QL in JPA?**

**Ans:**

```
String JPAQL=" from Customer cust where cust.email=?1";  
List<Customer> mylist= entityManager.createQuery(JPAQL)  
                                .setParameter(1, email)  
                                .getResultList();
```

**Q60) How to use JPA QL in JPA?**

**Ans:**

```
String SQL="select * from mycustomers where email=?";  
List<Customer> mylist= entityManager.createNativeQuery(SQL, Customer.class)  
                                .setParameter(1, email)  
                                .getResultList();
```

**Q61) How to use NamedQueries in JPA?**

**Ans:**

```
@NamedQuery(name = "customerByCity",  
            query = "from Customer cust where cust.city=:mycity")
```

```
List<Customer> mylist= entityManager.createNamedQuery("customerByCity")  
                                .setParameter("mycity", city)  
                                .getResultList();
```

**Q62) How to use Criteria Queries in JPA?**

**Ans:**

```
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();  
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);
```

```
Root<Customer> myroot = mycriteria.from(Customer.class);  
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), myemail);  
mycriteria.where(emailPredicate);
```

```
List<Customer> mylist=entityManager.createQuery(mycriteria).getResultList();
```



**Q63) What are the Log Levels /Priorities Available?**

**Ans:**

DEBUG, INFO, WARN, ERROR, FATAL

**Q64) Is logger creates any performance issue in PROD Environment?**

**Ans:**

Dev Mode : Use Debug

PROD Mode : INFO/WARN

**Q65) What happens when I set INFO level for com?**

**Ans:**

logging.level.com=INFO

Log Level for All the packages Starts with com will be set to INFO.

**Q66) How to use YAML Files (Yet another Markup Language)?**

**Ans:**

- Create the file called application.yml under src/main/resources
- Write the Required Properties in application.yml file

**Q67) How Spring Boot scans for application.priperities or application.yml?**

**Ans:**

Spring Boot scans for these files in the Classpath Resources.

If found in the classpath , You can see the following in Logs.

Origin: class path resource [application.yml]

Origin: class path resource [application.properties]

**Q68) Can I use both application.priperities and application.yml?**

**Ans:**

Yes, You can.

**Q69) How can I use Declarative Transactions in Spring Boot?**

**Ans:**

Nothing Special in Spring Boot. It is same as Spring Framework

**@Transactional**

Class AccountDAOImpl implements AccountDAO{

**@Transactional(propagation = Propagation.REQUIRED)**

public void fundsTransfer(int saccno, int daccno, double amount) {

//Core Logic Here

}

}



**Q70) How can I use Pragmatic Transactions in Spring Boot?**

**Ans:**

Nothing Special in Spring Boot. It is same as Spring Framework

```
TransactionStatus ts=null;
try {
    TransactionDefinition txDef = new DefaultTransactionDefinition(
        TransactionDefinition.PROPAGATION_REQUIRED);

    ts =txManager.getTransaction(txDef); //Tx Begin

    //Do your OPeratiosn Here

    txManager.commit(ts); //Tx Ends

} catch (Exception ex) {
    txManager.rollback(ts); //Tx Ends
}
```